

“We used ChatGPT to review our draft explanations”

IRWA Project Part-4: RAG, User Interface, and Web Analytics

Authors: Carla Núñez u214970 / Julia Pérez u214026 / Jan Prats u213927

GitHub URL: https://github.com/Januto30/SearchEngine-IRWA_2025

TAG: IRWA-2025-part-4

1. INTRODUCTION

This project presents a self-contained web application that serves as a robust demonstration of a modern search and analytics ecosystem. The primary goal is to provide a comprehensive platform where users can query a corpus of product documents and experience the full lifecycle of a search interaction, from query processing to result analysis.

Crucially, the platform elevates the standard search experience through a **Retrieval-Augmented Generation (RAG)** pipeline, which leverages the retrieved search documents to generate contextual, concise AI summaries for the user's query.

Beyond the core search functionality, the project incorporates a complete **Analytics Pipeline** that tracks and reports on user interactions. This pipeline logs essential metrics allowing for data-driven insights into document popularity, query frequency, and user behavior, all accessible via dedicated statistics routes and a dashboard.

2. NEW IMPLEMENTATIONS

New functions have been defined to handle the core operations.

web_app.py

The main Flask application that handles web routes (`/`, `/search`, `/doc_details`, `/dashboard`). It orchestrates the flow: accepting a search query, using `SearchEngine`, recording data via `AnalyticsData`, and rendering results.

index(): Handles the root URL (`/`). Initializes a session, logs user agent/IP, and renders the welcome page.

search_form_post(): Handles the search form submission (`/search`). Saves the query, executes the search via `SearchEngine`, stores results for analytics, generates a RAG response, and renders the results page.

doc_details(): Handles document clicks (`/doc_details`). Retrieves document details, records the click event in `AnalyticsData`, and renders the document detail page.

stats(): Displays simple click statistics. Fetches click counts from AnalyticsData, maps them to StatsDocument objects, and renders the stats page.

analytics_top_clicked(): Returns the top N clicked document PIDs and their counts.

analytics_top_queries(): Returns the top N most frequent search queries.

analytics_top_terms(): Returns the top N most frequent individual terms from all queries.

analytics_browsers(): Returns statistics on user browser usage from click events.

analytics_ips(): Returns statistics on user IPs, including geolocation data if available.

dashboard(): Renders the dashboard page, showing a list of clicked documents and their view counts.

plot_number_of_views(): Returns an HTML string for an Altair bar chart of document view counts.

settings(): Handles the /settings route. It allows the user to select the preferred ranking algorithm (ranking_method) and the desired RAG implementation (rag_impl)..

search_engine.py

Provides the search interface (SearchEngine class) that executes the search using the TF-IDF algorithm from algorithms.py. It also includes a dummy_search fallback for random results.

search(search_query, search_id, corpus, ...): Main search entrypoint. It supports multiple ranking backend methods: 'tfidf', 'bm25', 'our', or 'custom'. It directs the query to the appropriate cached retrieval method via algorithms.search_in_corpus. Results are converted from ranked (pid, score) tuples into a list of ResultItem objects.

SearchEngine (Class): Encapsulates the search engine logic and holds placeholders for cached index structures (like TF-IDF vectors).

analytics_data.py

Defines the AnalyticsData class, an in-memory persistence store for tracking user interactions, including search queries, click events, and generating statistical reports like top queries and browser usage.

AnalyticsData (Class): In-memory store for all analytics data (queries, search results, click counts, detailed click events).

save_query_terms(terms): Saves a new search query string, timestamps it, and returns a unique search_id.

save_search_results(search_id, results): Stores the PIDs, ranks, and positions of the documents returned for a specific search_id.

plot_number_of_views(): Generates an Altair bar chart visualization of document view counts and returns it as an HTML string.

get_top_queries(n): Returns the top n most frequently searched query strings.

get_top_terms(n): Returns the top n most frequent individual terms found across all queries.

get_browser_stats(): Parses user agents from click events and returns the counts per browser type.

get_ip_stats(): Returns counts per remote IP address, attempting to resolve and include city/country using GeoIP.

record_click(pid, search_id, ...): Increments the view counter for pid and records a detailed click event, including position and rank if a search_id is provided.

get_top_clicked(n): Returns the top n most clicked document PIDs and their counts.

_init_geo_reader(self): Attempts to initialize and return a GeoIP2 database reader.

_resolve_ip_to_location(self, ip): Resolves an IP address into a location dictionary using GeoIP2. Returns city, region, country, latitude, and longitude when available.

algorithms.py

The search algorithms have been modified so they no longer operate under a strict "all-or-nothing" match. If at least one document contains all query terms, those results are prioritized first. However, if no document includes every term, the system now falls back to partial matching, retrieving and ranking documents by the number and relevance of matched terms instead of returning an empty result. This change helps the system behave more like modern search engines, where partial, fuzzy or incomplete matches still return useful results rather than failing the query.

build_terms(text): Simple tokenizer: lowercases text, removes non-alphanumeric characters (except whitespace), and splits on whitespace.

create_tfidf_index_from_corpus(corpus): Builds a TF-IDF index. Calculates term frequency (TF) per document and inverse document frequency (IDF) globally. Returns document TF-IDF vectors and the global IDF map.

search_query_tfidf(query, doc_vectors, idf, top_k): Calculates the cosine similarity between the query's TF-IDF vector and all document TF-IDF vectors. Returns the top top_k results as (pid, score) tuples.

search_in_corpus(query, corpus, top_k): Main TF-IDF Search API. Handles caching of the TF-IDF index for the given corpus and calls search_query_tfidf to get ranked results.

create_bm25_index_from_corpus(corpus): Builds the necessary structures for the BM25 model, including the inverted index, document frequencies (df_counts), document lengths (doc_len), and the BM25 Inverse Document Frequency (idf_bm25). It also calculates the avgdl (average document length).

score_bm25_for_docs(query_terms, ...): Calculates the raw BM25 score for a set of matching documents against a list of query terms, using the formula dependent on term frequency (freq), document length (dl), and average document length (avgdl).

bm25_search(query, index, ...): Main search entry point for BM25. It tokenizes the query, determines the set of matching documents (preferring intersection but falling back to union), and uses score_bm25_for_docs to rank the results.

score_custom_for_docs(query_terms, ..., corpus): Implements a Hybrid Scoring Model. It uses the raw BM25 score as a base and then applies a feature boost based on document metadata (e.g., average_rating, out_of_stock status, selling_price, and discount). The boosts are normalized and weighted to influence the final ranking.

rag_enhanced.py

We were given a basic Retrieval-Augmented Generation (RAG) pipeline that takes a query and retrieves product information from the e-commerce dataset to generate answers for users.

We analyzed the weaknesses of the baseline RAG and proposed several ways to improve it:

- Adding structured I/O: mandates and expects a strict JSON output (keys: best_pid) so downstream code can parse reliably. rag.py returns free-form text (a string) and leaves parsing to callers.
- Richer retrieved-result context: formats retrieved items as "Rank, PID, Score, Title, Desc" (via _format_results), it exposes rank and score in the prompt. rag.py only injects PID and Title lines.
- Robust parse/validation: After the LLM returns, rag_enhanced.py extracts JSON and checks best_pid against the retrieved PID set; if the model proposes a PID not in the retrieval set it falls back to a deterministic top-ranked choice and annotates notes/llm_raw. rag.py does no such validation.
- Deterministic fallback paths: supports mode='deterministic' and also falls back to deterministic behavior automatically when the LLM client (Groq) is unavailable or when parsing fails. rag.py attempts the LLM call and returns a default string only on exception.
- Safer error handling: prints/returns diagnostic info (e.g., llm_raw when falling back) and sets notes describing the fallback reason. rag.py prints exceptions and returns a generic default string on failure.

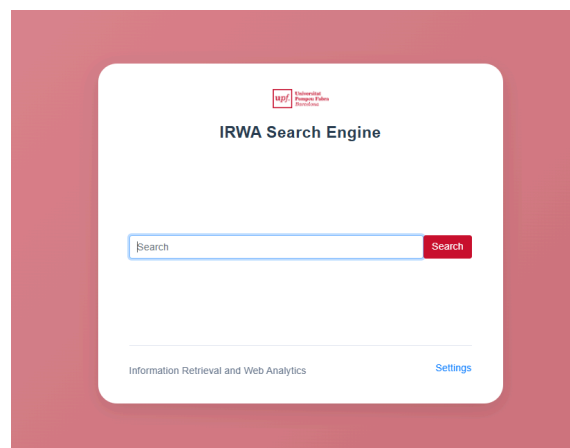
3. GRAPHICAL USER INTERFACE AND USER EXPERIENCE

The Graphical User Interface (GUI) is designed for efficiency and clarity, guiding the user through the search, retrieval, and analysis process. The application follows a clean, three-part structure: the main search page, the results display, and detailed document views, all unified under a consistent aesthetic.

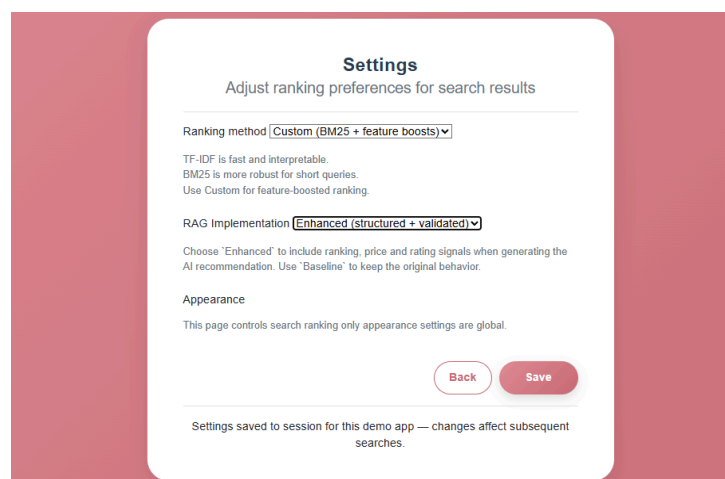
3.1. USER INTERFACE FLOW

The user journey begins at the Home Page (index.html), which features a centralized search box for immediate query submission.

Search Results (results.html): After submitting a query, the user is presented with a list of documents ranked by relevance, along with the total number of results found. Each result item displays the document's title, a brief description, and metadata like price, discount, and average rating (if available). Critically, each item's ranking score is displayed.



Settings Page (settings.html): The system includes a dedicated settings page to allow users to dynamically select the search and RAG algorithms. This form lets the user choose between multiple Ranking Methods for retrieval: TF-IDF (cosine), BM25, or Custom (BM25 + feature boosts). Additionally, the user can select the desired RAG Implementation (Baseline or Enhanced). These choices are persisted in the user's session and affect all subsequent searches.



RAG Integration: A key feature on the results page is the AI-Generated Summary (RAG response), which provides a concise, contextual answer to the query based on the retrieved documents, enhancing information synthesis for the user.

 **UPJ** Universitas
Pangrehareja
Jember

IRWA Search Engine

Found 20 results...

AI-Generated Summary

- Best Product: SHTFZTHMB3BW8HHHD Women Slim Fit Printed Slim Collar Casual Shirt

- Why: This product is the best fit for the user's request as it exactly matches the criteria of "women slim fit," including the specific style details such as slim collar and casual fit. It also offers a printed design option, which suits someone looking for variety in fashion.

- Alternative: SHTF5ZG9Z2H4ZRE9 Women Slim Fit Printed Slim Collar Casual Shirt could be considered as an alternative, as it also meets the user's requirements with slight variations in available design options.

Women Slim Fit Solid Slim Collar Casual Shirt

Price: 606.0 — Discount: 75.0% — Rating: 3.0/5

this cotton fabric shirt

Score: 0.5889

Women Slim Fit Printed Casual Shirt

As a demonstration, by entering a query (for example: "war") where no document contains the word in the title but it does in another attribute of the document.

 **UPJ** Universitas
Pangrehareja
Jember

IRWA Search Engine

Found 5 results...

AI-Generated Summary

- Best Product: SMSF5R7F2ZYCYKH

- Why: The highest-ranked product with the word 'war' is the Full Sleeve Graphic Print Women Sweatshirt, with a score of 0.36108832684753867 and a descriptive title. It features a character from the Civil War.

Printed Men Round Neck White T-Shirt

Price: 450.0 — Discount: 65.0% — Rating: 3.0/5

Refresh your clothing with the awesome collection of Round Neck Tees from T-SHIRT EXPRESS These t shirts are made of 100% Bio wash Cotton and make a comfort wear for all seasons. The feel of the fabric keeps you comfortable even at high humid conditions. Pair it with Cotton Trousers or Denims for a perfect weekend wear.

Score: 0.4096

[Home](#) | [Stats](#) | [Dashboard](#)

Printed Men Round Neck White T-Shirt



Price: 450.0 (Discount: 65.0%)

Rating: 3.0

Description

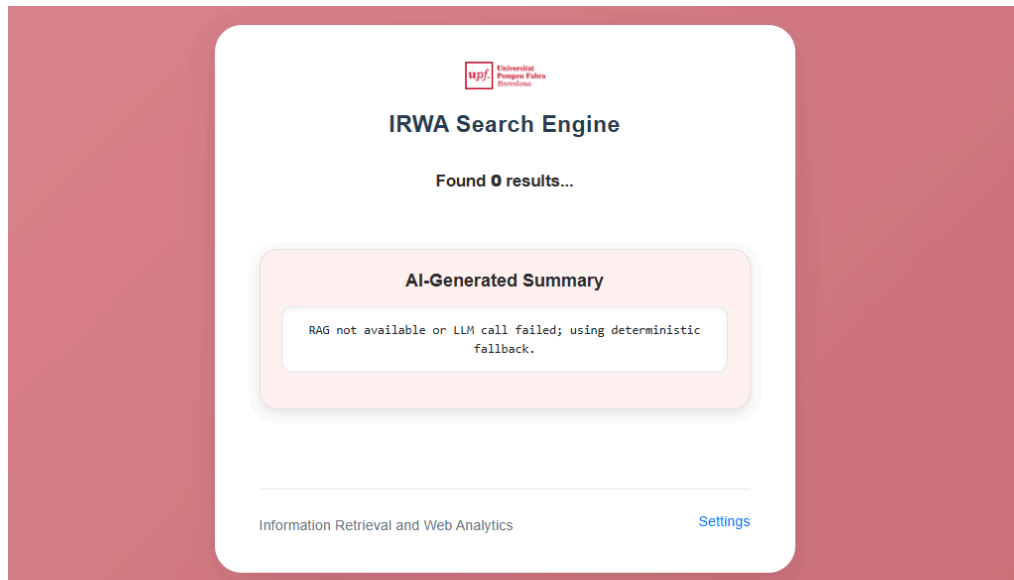
Refresh your clothing with the awesome collection of Round Neck Tees from T-SHIRT EXPRESS These t shirts are made of 100% Bio wash Cotton and make a comfort wear for all seasons. The feel of the fabric keeps you comfortable even at high humid conditions. Pair it with Cotton Trousers or Denims for a perfect weekend wear.

Product Details

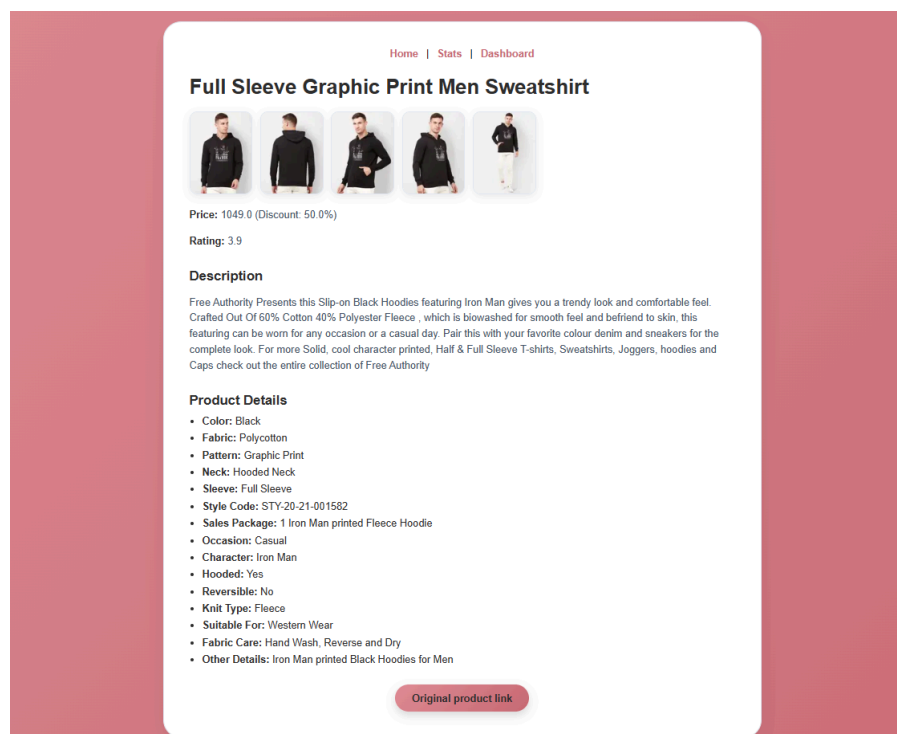
- Type: Round Neck
- Sleeve: Short Sleeve
- Fit: Slim
- Fabric: Pure Cotton
- Sales Package: 1
- Pack of: 1
- Style Code: CAPTAIN AMERICA CIVIL WAR WHITE COTTON T SHIRT
- Neck Type: Round Neck
- Ideal For: Men
- Size: M
- Pattern: Printed
- Suitable For: Western Wear
- Brand Fit: SLIM
- Sleeve Type: Narrow
- Reversible: No
- Character: Captain America, Iron Man
- Fabric Care: Regular Machine Wash
- Model Name: CAPTAIN AMERICA CIVIL WAR T SHIRT
- Brand Color: WHITE
- Generic Name: T Shirts
- Country of Origin: India

[Original product link](#)

No Results/RAG Fallback: If a search yields zero results the system handles the situation. The results page will display "Found 0 results..." and the AI Summary box will show a deterministic fallback message such as "RAG not available or LLM call failed; using deterministic fallback." to inform the user of the status.



Document Details (doc_details.html): Clicking a search result links to the detail page for that document. This view displays the full product information, including the document title, description, detailed product specifications, images, price, and rating. Crucially, this action simultaneously triggers a click recording in the analytics backend.



3.2. RAG

The project employs a sophisticated Retrieval-Augmented Generation (**RAG**) pipeline, encapsulated in the **RAGEnhancedGenerator** class, to transform raw search results into concise, actionable, and context-aware summaries. This component is crucial for enhancing the user experience by providing synthesized answers rather than just a list of links.

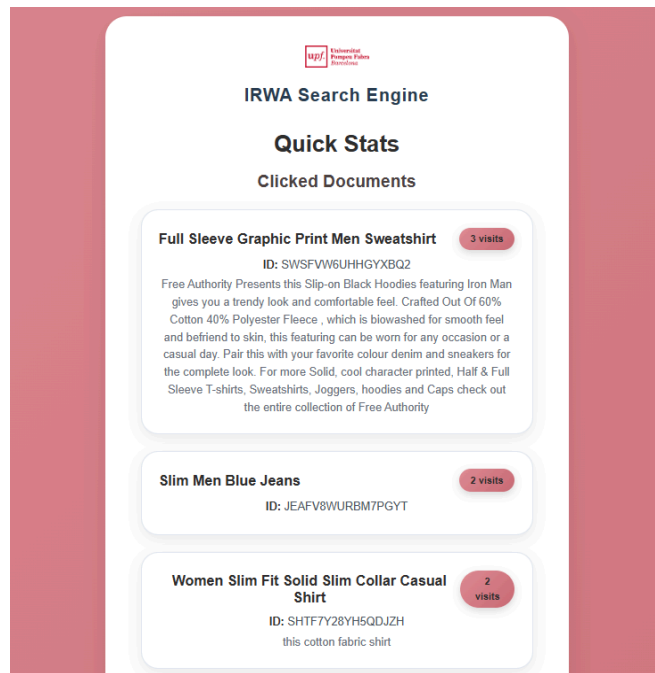
The **RAGEnhancedGenerator** serves as the interface between the search core and the LLM (using the Groq API). It is designed to overcome common LLM limitations by emphasizing structured output and robust failure handling. The following table details the purpose of the key components within this generator:

Class/Method	Purpose
PROMPT_TEMPLATE	Defines the instruction set for the LLM. The prompt acts as an expert product recommender, explicitly requesting a JSON output with specific keys (best_pid, why, alternative) and providing ranking rules that prioritize items based on retrieval score and product attributes (price, rating).
_format_results()	Prepares the context for the LLM. It formats the list of retrieved documents into a clean string, including the Rank, PID, Score, Title, and a short Description for the top N results.
generate_response()	Main entrypoint for RAG. It constructs the full prompt and calls the external Groq API. It supports switching between augment mode (uses the LLM) and deterministic mode (skips the LLM, using the fallback).
_extract_json_from_text()	A custom parsing utility that attempts to robustly extract the required JSON object even if the LLM output contains surrounding natural language or formatting errors.

3.3. WEB ANALYTICS AND DASHBOARDS

The system provides two distinct visualization endpoints for system performance and user behavior analysis:

Quick Stats (stats.html): Offers a high-level, list-based view of the most Clicked Documents, showing the document title, ID and the total count of visits in a clean format.



Dashboard (dashboard.html): Presents a comprehensive, graphical overview of the system's usage.

This page uses Charts.js to visualize key metrics retrieved via dedicated API endpoints:

- **Ranking of Visited Documents:** Horizontal bar chart showing clicks per product ID.
- **Visitor's Preferred Browsers:** Pie chart illustrating browser distribution based on user agents.
- **Query Analysis:** Bar charts displaying rankings of the most frequent search queries and most frequent individual terms.
- **Visitor's IPs:** A table listing the source IPs, including city and country, and the corresponding click counts.



This dual approach ensures both immediate statistical reporting and detailed graphical analysis of the IRWA ecosystem.

Note: To obtain the city and country of the user accessing the web application, follow the steps described in the project's [README.md](#) file.

The IP addresses displayed when running *web_app.py* (127.0.0.1 and 192.168.x.x) are private or local addresses, used only within the development internal network. These IPs are not found in geolocation databases (GeoIP) and therefore cannot be used to obtain city or country information.

We haven't been able to get the city and country to display on the website. The procedure we followed to try to configure it, is what you will find in the README file.